Cross-Domain 2D Object Detection: A Post-Processing Failure Audit

Harshit Jangam | CMPE 249 | Experimental Track

Abstract

This report evaluates four COCO-pretrained 2D object detector configurations without fine-tuning on a shared three-class protocol: person, car, and truck. The source domain is COCO val2017 and the target domain is KITTI. Rather than presenting one aggregate score, the study combines cross-domain metrics, scale and class breakdowns, latency and memory profiling, TIDE error decomposition, and a fixed-weight NMS-IoU intervention. All reported metrics come from deterministic 1,000-image saved prediction sets. The result is a controlled failure audit, not a claim of a new detector or state of the art.

1. Question and hypothesis

The primary question is: when a COCO-pretrained detector is transferred to an autonomous-driving domain without retraining, how much of the performance loss can be attributed to final duplicate suppression rather than upstream localization, background, classification, or recall failures? The testable hypothesis is that changing only final-stage NMS should affect duplicate errors and may recover some accuracy, but cannot remove upstream failure modes.

2. Experimental protocol

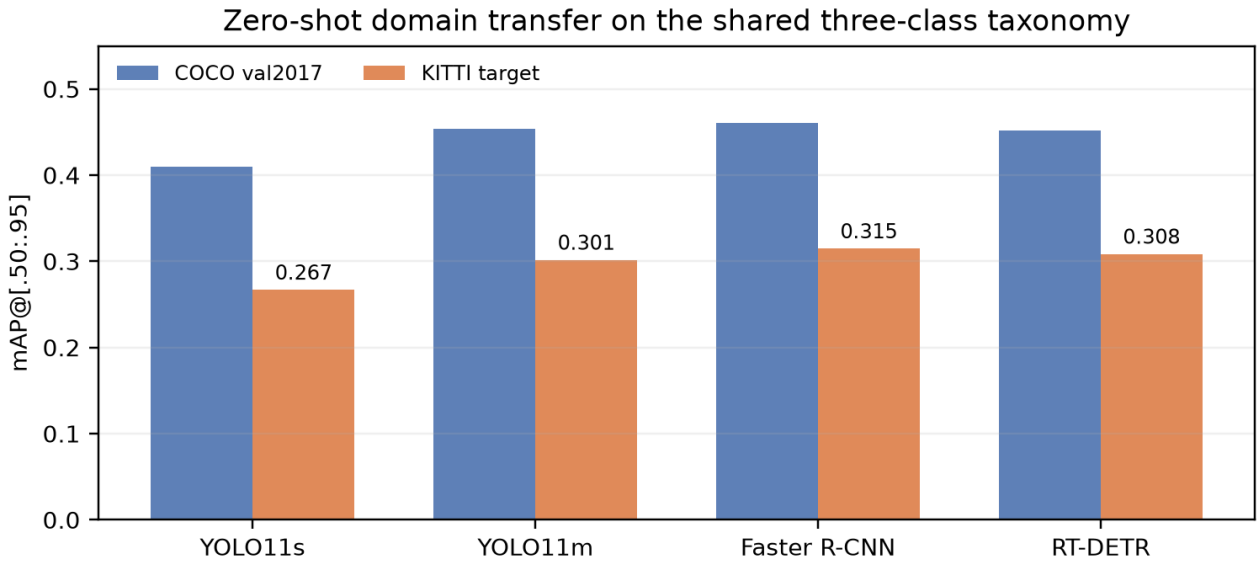
Four configurations span three detector families: YOLO11s and YOLO11m are one-stage dense-prediction models with NMS; Faster R-CNN R50-FPN v2 is a two-stage region-based detector with final RoI NMS; and RT-DETR R50 is a query-based, NMS-free detector. COCO val2017 and KITTI target images are filtered to person, car, and truck. COCO-only and KITTI-only labels are excluded rather than heuristically remapped. Each run evaluates a deterministic prefix of 1,000 images, uses a score floor of 0.001, saves at most 300 candidates, and evaluates at most 100 detections per image. Object-scale labels use original annotation coordinates, not post-resize coordinates.

Baseline measurements were run on a single RTX A6000 48 GB cloud GPU. The later NMS thresholds were executed on an RTX 4090 24 GB after recovery. Therefore, only baseline latency and memory values are used for cross-model efficiency comparison; NMS sweep latency is not compared across the two GPUs.

Configuration	Family	Input preprocessing	Baseline post-processing
YOLO11s	one-stage dense	letterbox 640 x 640	final class-aware NMS IoU 0.70
YOLO11m	one-stage dense	letterbox 640 x 640	final class-aware NMS IoU 0.70
Faster R-CNN R50-FPN v2	two-stage	min side 800; max side 1333	RoI NMS IoU 0.50; RPN fixed
RT-DETR R50	query-based	processor resize 640 x 640	set prediction; no final NMS

The KITTI reference image size is 1242 x 375. The Faster R-CNN transform therefore scales the image subject to the 800-pixel short-side and 1333-pixel long-side bounds, while the two YOLO configurations and RT-DETR receive 640 x 640 tensors. This is intentionally a checkpoint-native comparison, not a resolution-matched comparison. Score floor is 0.001 for all models; it preserves the low-score tail needed for the precision-recall curve. The baseline NMS settings are independently validated by the sweep: the baseline values land between their adjacent sweep values.

3. Baseline cross-domain evaluation


Zero-shot domain transfer on the shared three-class taxonomy
COCO val2017
KITTI target
mAP@[.50:.95]
0.0
0.1
0.2
0.3
0.4
0.5
0.267
0.301
0.315
0.308
YOLO11s
YOLO11m
Faster R-CNN
RT-DETR

3. Baseline cross-domain evaluation (continued)

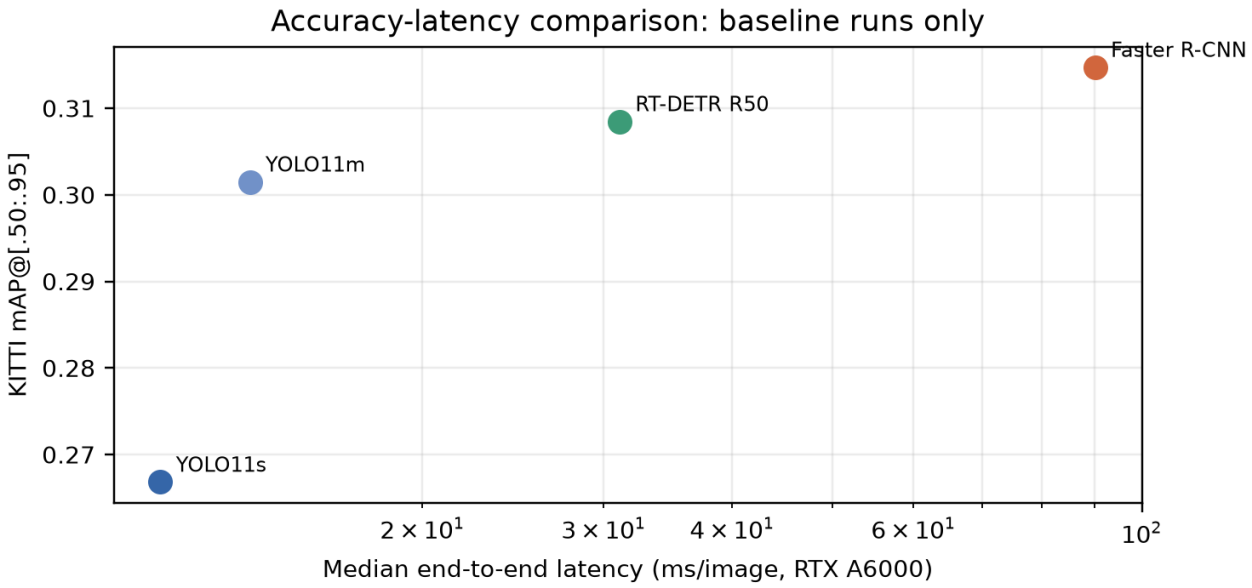
Figure 1 shows a consistent transfer gap. The target-domain mAP decreases by 0.143 for YOLO11s, 0.153 for YOLO11m, 0.146 for Faster R-CNN, and 0.144 for RT-DETR. Faster R-CNN has the highest baseline KITTI mAP (0.315), narrowly ahead of RT-DETR (0.308) and YOLO11m (0.301). The result does not establish an architecture as universally better: it is bounded to this three-class, zero-shot, 1,000-image protocol.

Model	COCO mAP	KITTI mAP	AP50	AP75	AP S/M/L	person/car/truck AP
YOLO11s	0.410	0.267	0.483	0.271	0.176/0.257/0.380	0.247/0.446/0.107
YOLO11m	0.454	0.301	0.536	0.312	0.180/0.292/0.423	0.256/0.463/0.186
Faster R-CNN	0.460	0.315	0.555	0.334	0.161/0.311/0.420	0.260/0.473/0.211
RT-DETR R50	0.452	0.308	0.531	0.315	0.217/0.303/0.426	0.258/0.492/0.175

Across all four target-domain models, cars are substantially easier than persons and trucks. The lowest target AP values occur for YOLO11s truck (0.107) and YOLO11s small objects (0.176). The scale pattern is also consistent: large objects are easier than medium, which are easier than small, except RT-DETR's target small-object score (0.217) is comparatively stronger than the other configurations. These observations satisfy the cross-dataset and scale-conditioned analyses without implying that the finite target subset estimates the complete KITTI benchmark.

The course ngdet tutorial reports the same broad pattern on its separate 120-image driving3 evaluation: Faster R-CNN 0.279, RT-DETR 0.262, and YOLO11x 0.260 on KITTI [10]. The protocols differ in taxonomy, model sizes, image count, score floor, and GPU, so these values are not a numerical reproduction. However, both studies show cross-domain compression: the present COCO-to-KITTI mAP drops span only 0.143 to 0.153 across three detector families. This audit extends that teaching observation by testing whether final-stage NMS accounts for a material portion of the residual gap.

4. Architecture and efficiency comparison

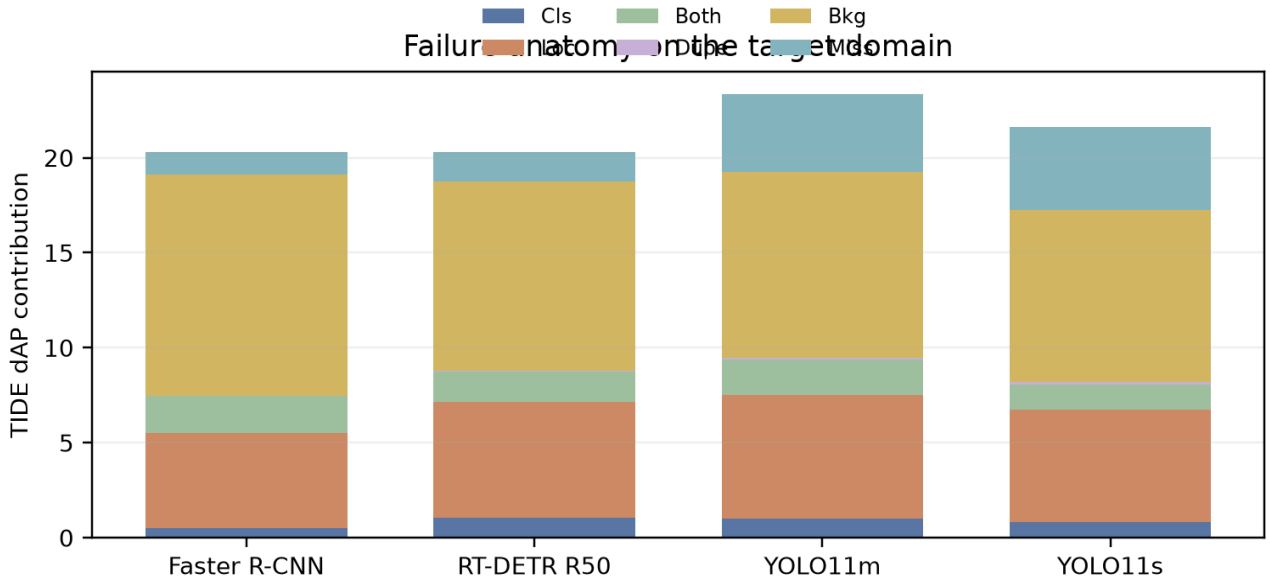


The baseline A6000 timing study shows the expected practical trade-off. YOLO11s is fastest at 11.1 ms/image (about 90 FPS) and uses 59 MiB peak allocated CUDA memory, but has the lowest target mAP. Faster R-CNN is most accurate in this protocol but has 90.1 ms median latency and 439 MiB peak allocated memory. RT-DETR is NMS-free at inference and has 31.1 ms median latency with 298 MiB allocated memory. All timings include model preprocessing and post-processing in the same single-image evaluation harness; they are not vendor benchmark claims.

Model	KITTI mAP	AR@100	Median ms	FPS	Peak MiB
YOLO11s	0.267	0.462	11.1	87.4	59
YOLO11m	0.301	0.501	13.6	71.5	115
Faster R-CNN	0.315	0.581	90.1	10.9	439
RT-DETR R50	0.308	0.584	31.1	31.4	298

The comparison also clarifies what is and is not held constant. Architecture-specific preprocessing is preserved because forcing a two-stage detector into a square letterbox would introduce a confound. Evaluation annotations, taxonomy, score floor, candidate cap, metric cap, image manifest, batch size, and baseline GPU type are held fixed. The efficiency analysis is therefore a practical system comparison, not a claim that all architectures receive identical internal feature resolution.

5. Failure anatomy

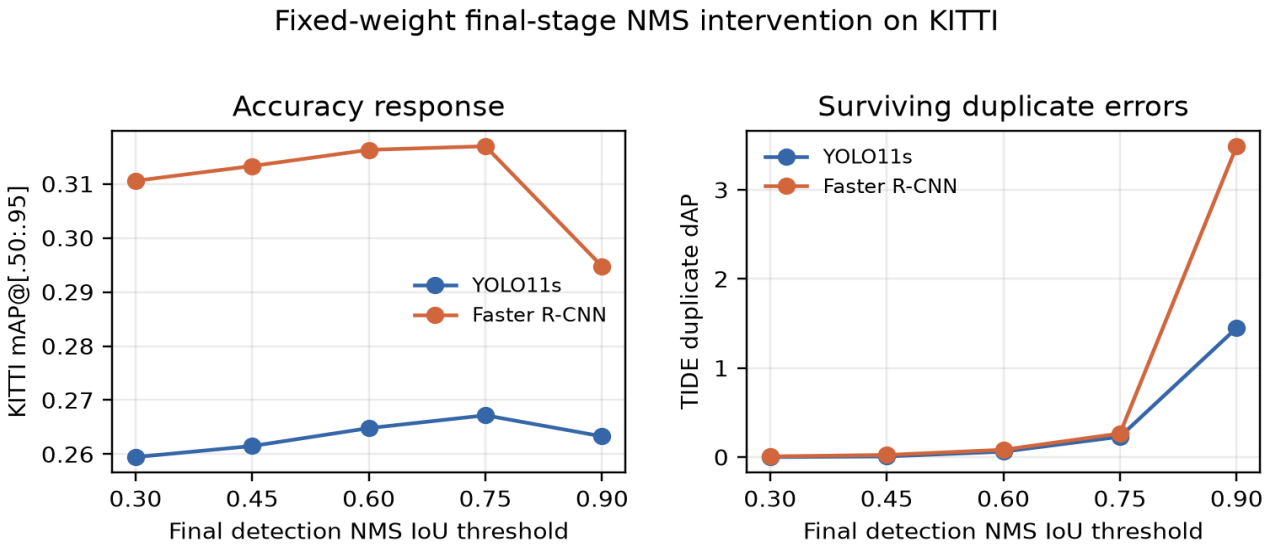


TIDE separates error contributions after prediction. On KITTI, background and localization errors dominate the visible error anatomy, while duplicates are comparatively small at the baseline thresholds. This motivates the intervention instead of assuming that NMS is the root cause. TIDE duplicate dAP is observational: it measures errors that survived post-processing, not a direct causal estimate of NMS behavior.

Target-domain ground truth stratum	Count	Definition
small / medium / large	790 / 2,416 / 1,281	COCO area bands in original image coordinates
same-class isolated / near / overlap	2,579 / 1,102 / 806	max same-class IoU < .10 / [.10, .30) / >= .30
all-class isolated / near / overlap	2,472 / 1,187 / 828	same bands using any-class spatial neighbor
person / car / truck	646 / 3,685 / 156	strict shared taxonomy only

The density strata distinguish same-class overlap from any-class neighborhood density. That distinction matters because class-aware NMS acts within a class, whereas cross-class proximity can still cause occlusion and feature ambiguity without being directly suppressed. The target set contains relatively few trucks, so truck conclusions should be interpreted as evidence from the declared fixed subset rather than a broad population estimate.

5. NMS intervention



The causal test holds model weights, images, score floor, and saved-candidate cap fixed while sweeping only final detection NMS IoU. The largest observed point estimates occur at 0.75: 0.267 for YOLO11s and 0.317 for Faster R-CNN. However, the differences from 0.60 are only 0.0024 and 0.0007, respectively, and this study does not estimate confidence intervals. The correct interpretation is a flat intermediate-threshold plateau, not a resolved optimum. Both models fall at 0.90 while duplicate dAP rises sharply. For Faster R-CNN, RPN proposal NMS remained fixed, isolating final RoI suppression. Thus a looser threshold can recover some overlapping detections, but excessive permissiveness increases surviving duplicates and does not solve background or localization errors.

Model	NMS 0.30	0.45	0.60	0.75	0.90
YOLO11s	0.259	0.262	0.265	0.267	0.263
Faster R-CNN	0.311	0.313	0.316	0.317	0.295

6. Limitations and conclusion

This is a zero-shot study, not a fine-tuning study. It uses a strict shared taxonomy, so non-target classes are excluded from the three-class evaluator. In particular, KITTI `Van`, `Cyclist`, `Person_sitting`, and `Tram` are not relabeled. This prevents a synthetic cyclist class, but means the analysis does not measure open-set recognition. COCO crowd annotations in the target classes were dropped rather than retained as COCO ignore regions; accordingly, results are strict three-class audit metrics and are not comparable to official full-category COCO AP. A sensitivity re-audit with retained ignore regions would require the original annotation files because discarded boxes are not present in the saved records.

Native preprocessing differs by model family and is documented rather than artificially forced to a single resize recipe; therefore the accuracy-latency comparison is checkpoint-native, not resolution-matched. Unlike the official KITTI benchmark, this audit scores every mapped object regardless of truncation, occlusion, or box height, so small-object results are intentionally not comparable to KITTI leaderboard difficulty tiers. The truck class has only 156 target instances and COCO/KITTI truck definitions do not align perfectly. The deterministic 1,000-image subsets support reproducibility and matched comparisons, but are not official full-dataset leaderboard evaluations. Finally, the NMS sweep and baseline speed study use different GPU models, so the report keeps their interpretations separate.

The main conclusion is that the cross-domain drop is not reducible to duplicate suppression. Across the measured threshold range, YOLO11s gains 0.77 mAP points and Faster R-CNN gains 0.64 points from NMS 0.30 to 0.75, versus COCO-to-KITTI drops of 14.29 and 14.55 points. This bounds the observed final-stage NMS effect to about five percent of the domain gap in this protocol. The stronger target-domain models still exhibit substantial background and localization contributions. A useful next experiment would fine-tune one configuration on the target training split and repeat the same TIDE and NMS protocol with bootstrap confidence intervals and explicit ignore-region sensitivity analysis.

Evidence and reproducibility

The submitted repository includes versioned configurations, exact cloud commands, evaluation code, compact summary tables, and figure-generation code. Raw predictions and images are intentionally excluded from Git because they are large; each saved run has a SHA-256 hash recorded in the compact summary. The report is generated by scripts/analysis/build_submission.py directly from those saved artifacts.

References

[1] S. Ren et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NeurIPS, 2015.

[2] N. Carion et al. End-to-End Object Detection with Transformers. ECCV, 2020.

[3] Y. Zhao et al. DETRs Beat YOLOs on Real-time Object Detection. CVPR, 2024.

[4] D. Bolya et al. TIDE: A General Toolbox for Identifying Object Detection Errors. ECCV, 2020.

[5] T.-Y. Lin et al. Microsoft COCO: Common Objects in Context. ECCV, 2014.

[6] A. Geiger et al. Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite. CVPR, 2012.

[7] Ultralytics. YOLO Documentation. Official documentation, https://docs.ultralytics.com/.

[8] PyTorch Contributors. Torchvision Object Detection Models and Weights. Official documentation, https://pytorch.org/vision/stable/models.html.

[9] Hugging Face. Object Detection Task Guide. Official documentation, https://huggingface.co/docs/transformers/tasks/object_detection.

[10] lkk688. DeepDataMiningLearning ngdet Tutorial. Course starter repository, https://github.com/lkk688/DeepDataMiningLearning.